architecture.md

> **CivicSync · System Architecture** Version 1.0-rc1 · 28 Oct 2026 · Owner: Backend Lead (Manindra Shukla) · Co-owner: CV Lead (Aayush) Why we chose each piece is in `memory.md` (D-xx). Numbers (weights, thresholds, deadlines) are in `rules.md`.

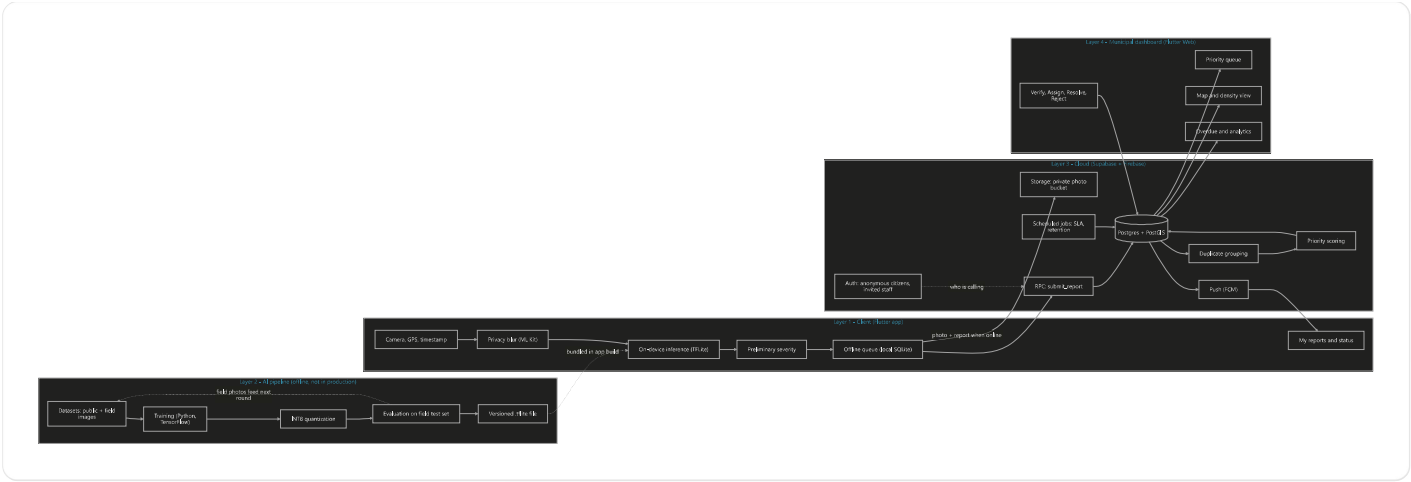
1. The idea in one paragraph

The citizen's phone does the *thinking that has to be fast and offline*: take the photo, find the place, guess the hazard, work out a starting severity, blur faces. The cloud does the *thinking that needs a shared view*: put the report in the right ward, notice that three other people reported the same pothole, and rank the ticket against everything else. The dashboard is where a person decides and acts. Four layers, one direction of travel, one feedback loop back to the citizen.

2. The four layers

Layer	What lives here	Main technology	Runs where
1. Client	Camera, GPS, privacy blur, TFLite inference, preliminary severity, offline queue, "My reports"	Flutter (Android first) · `camera`, `geolocator`, `tflite_flutter`, ML Kit, local SQLite	Citizen's phone
2. AI	Data curation, training, INT8 quantization, evaluation, model versioning	Python · TensorFlow/Keras · Colab or a university lab GPU	Team laptops and Colab (not in production)
3. Cloud	Auth, private photo storage, Postgres with PostGIS, duplicate grouping, priority scoring, scheduled jobs, push	Supabase (primary) · Firebase Cloud Messaging for push and Crashlytics	Managed services
4. Municipal	Priority queue, map, status actions, overdue list, analytics	Flutter Web (**D-04**) · Google Maps for the base map	Staff browsers

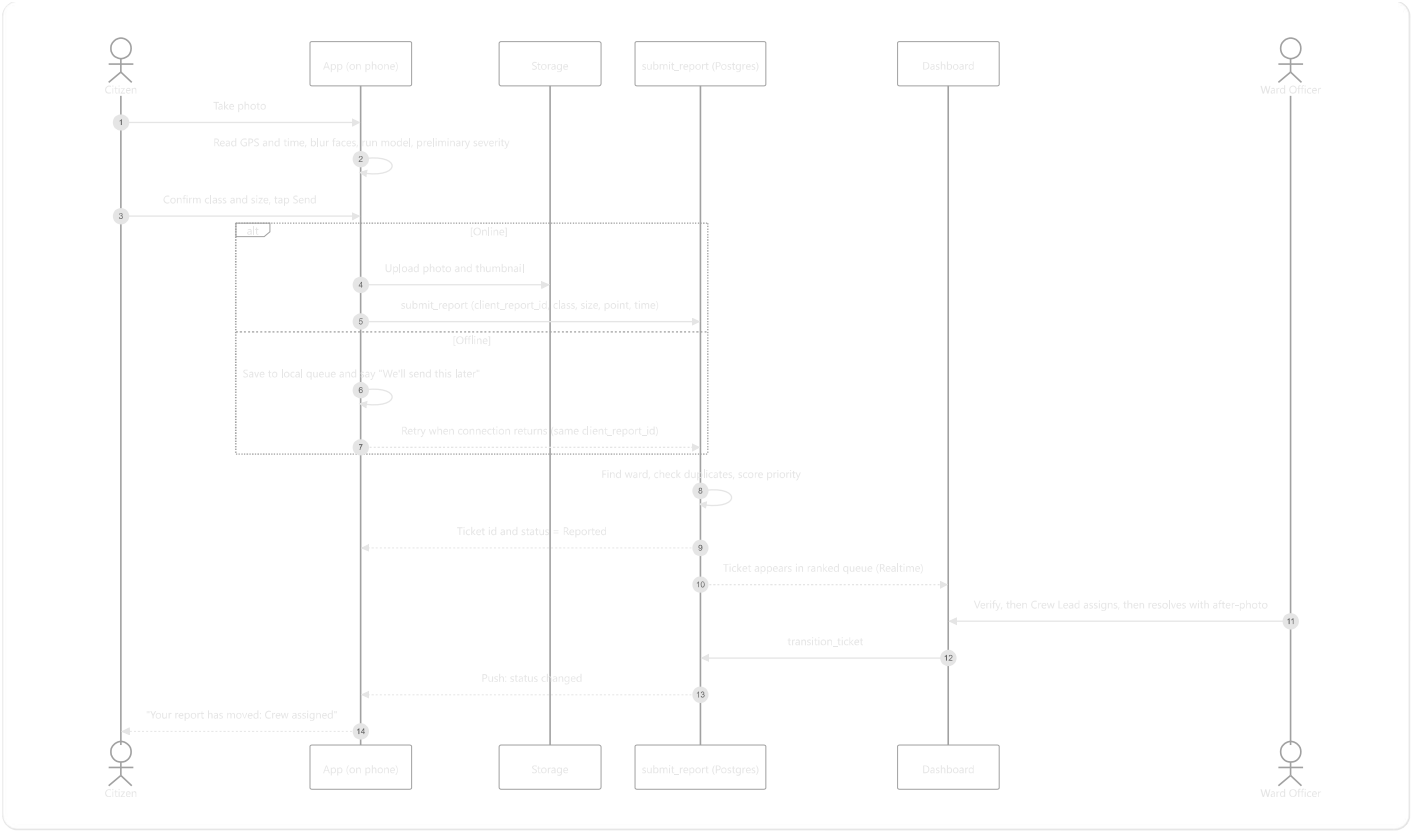
2.1 Component diagram



3. Data flow, step by step

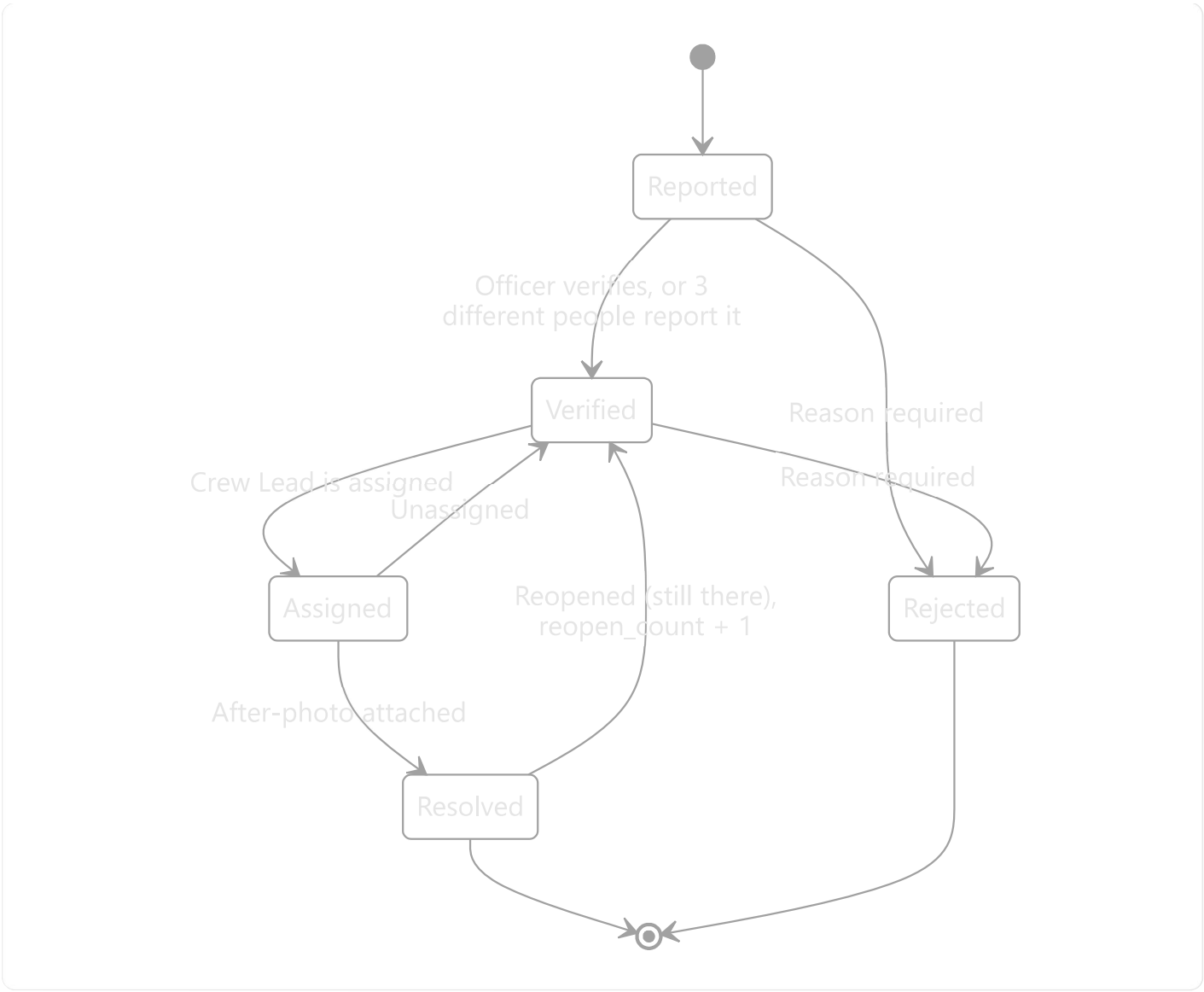
The proposal's eight steps, with where each one runs.

Step	Name	Runs on	What happens
1	Capture	Phone	Person opens the app and photographs the hazard. The photo is resized (long edge ⩽ 1280 px).
2	Locate	Phone	GPS position, accuracy and time are read at the moment of capture. Mock-location flag is read if the phone exposes it.
3	Classify	Phone	Face/plate blur runs, then the model returns a class and a confidence. The person can correct the class and pick a size.
4	Score	Phone	Preliminary severity from class, size and confidence (location context is set to "unknown", **D-10**).
5	Submit	Phone → Cloud	Photo and thumbnail upload; then `submit_report`. If offline, everything waits in the local queue.
6	Deduplicate	Cloud	Ward is found; nearby open tickets are checked; the report joins one or becomes a new ticket; priority is recalculated.
7	Act	Dashboard	Officers see a ranked queue and verify, assign and resolve.
8	Notify	Cloud → Phone	Status changes are pushed and appear in "My reports".



4. The status workflow

The citizen-facing words are in `design.md` §6. In the database the five values are `reported`, `verified`, `assigned`, `resolved`, `rejected`.



Guards (enforced in the database function `transition_ticket`, not only in the UI):

- Only roles allowed by `security.md` §4 can make a given change, and only inside their wards.
- `Resolved` requires an after-photo.
- `Rejected` requires a reason from the fixed list in `rules.md` §3.4.
- Every change writes a row in `ticket_events`.
- Reopening happens when the SLA-free "Still there?" check (`rules.md` §3.6) collects two confirmations, or when a Ward Officer reopens it.

---

5. Duplicate clustering

Goal: one real hazard, one ticket, however many people photograph it.

When: inside `submit_report`, after the ward is found, in one database transaction.

Inputs: the new report's point, its final class, its GPS accuracy.

Steps:

1. **Lock the neighbourhood.** Take a transaction-level advisory lock on a coarse grid cell of the point, so two reports arriving in the same second can't both create "new" tickets.
2. **Find candidates.** Select open tickets (`reported`, `verified`, `assigned`) whose class is in the same *compatibility group* and whose point is within 50 m. Compatibility groups: {pothole, broken_road}, {open_drain}, {waterlogging}, {other, only if the notes match or an officer chooses}.
3. **Decide by distance** (numbers from `rules.md` §6):
   - Nearest candidate ⩽ **25 m** → attach the report to it ("auto-merge").
   - Nearest candidate 25–50 m → create a new ticket and write a row in `duplicate_candidates` for an officer to review.
   - No candidate → create a new ticket.
4. **Same-person rule.** Several reports from the same account within 24 hours at the same spot count once toward the corroboration count.
5. **Recently fixed spots.** If a ticket in the same group was `resolved` within the last 14 days within 25 m, flag the new report as a possible *reopen* instead of silently opening a fresh ticket.
6. **Recompute.** Update `report_count`, `reporter_count` and the ticket's point (centroid of its reports that have usable GPS accuracy), then run the priority function.

```sql
-- Sketch only. Real functions live in /supabase/migrations and are tested with pg
select t.id, ST_Distance(t.geom, new_point) as d
from tickets t
where t.status in ('reported','verified','assigned')
  and same_group(t.class, new_class)
  and ST_DWithin(t.geom, new_point, 50)      -- geography: metres
order by d asc
limit 1;
```

Why distance and class, and not image similarity? It is cheap, explainable to an officer, works with the free-tier database, and doesn't need a second model on the server. An image-similarity check is a possible later upgrade (**D-11**).

Known weak spots: GPS drift in narrow lanes and under flyovers (we show low-accuracy points a pin to nudge); two real potholes 20 m apart merged wrongly (officers can split); long

cracks that stretch across many metres (we accept several tickets).

---

6. Triage and priority logic

Triage happens in two moments.

On the phone (Step 4): preliminary severity. Uses class weight, size and model confidence. Location context is "unknown" at this point, because road type and school zones need map data the phone doesn't carry in v1 (**D-10**).

In the cloud (Step 6): Priority Score. Re-uses the same formula with the *real* location context, then adds a small bonus when several different people reported the same hazard.

```
PriorityScore = round( 100 × ( 0.30·C + 0.25·E + 0.15·K + 0.30·L ) ) + B
(capped at 100)

C = class weight          E = extent (size)          K = model confidence
L = location context      B = corroboration bonus
```

The tables for C, E, L, B and the four bands (Critical, High, Medium, Low) are the single source of truth in `rules.md` §1. This file doesn't repeat them, so they can't drift.

Location context lookup: for the ticket's point, find the closest relevant rows in `context_features` (schools and hospitals within 200 m; primary and secondary roads within 30 m; markets, bus stands and rail stations within 200 m) and take the highest matching value. The features are imported from OpenStreetMap on a schedule (`tooling.md` §5).

Ranking in the queue: by `priority_band`, then `priority_score` descending, then oldest `first_reported_at`, so a long-waiting ticket doesn't sink.

---

7. Heatmaps without the deprecated layer

The dashboard's "where do hazards cluster?" view does **not** use the Google Maps JavaScript Heatmap Layer. Google's deprecation page says that layer was deprecated in May 2025 and became unavailable in May 2026 (**C-13**). Instead, the database groups open tickets into square grid cells (e.g. 250 m) using `ST_SnapToGrid` on projected coordinates, returns `{cell_center, count, max_priority}`, and the dashboard draws shaded circles or squares. It's also cheaper and easy to test.

---

8. Security and privacy in the architecture

Short version; details in `security.md`.

- Photos live in a **private** bucket; staff see them through short-lived signed links, and only for tickets in their wards.
- Row-level security is on for every table. The "service role" key lives only in server-side jobs, never in an app or dashboard build.
- Citizens are pseudonymous: no name, email or phone is needed to report.
- Retention jobs (`pg_cron`) delete or thin photos on the schedule in `rules.md` §7.

---

9. Failure modes and fallbacks

What goes wrong	What the person sees	What the system does
No signal	"Saved. We'll send this when you're back online."	Queue with idempotent retries (TRD-O2–O5).
GPS slow or weak	"We couldn't pin this down exactly. Drag the pin to the spot."	Marks `low_gps`; officer sees the flag.
Model unsure	"Not sure what this is. Pick the closest one."	Model guess is stored but not pre-selected.
Model file fails to load	Same as above	App works without suggestions.
Photo upload fails halfway	Nothing, it retries	Re-uses an uploaded photo if one exists.
Database paused (free plan)	"We're having trouble. Your report is saved."	Prevented by Pro plan before beta and a keep-alive ping before then.
Duplicate lands at the same instant	Nothing	Advisory lock avoids two tickets.
Officer doesn't act	Citizen sees "Checked" or "Reported" for long	SLA clocks and escalation (`rules.md` §4–5).
Maps quota exceeded	Map shows a grey grid, reports still work	Budget alerts at 50% and 80%; OpenStreetMap-based fallback map for the dashboard.

---

10. Deployment shape (pilot)

One Supabase project per environment (`dev`, `prod`), one Firebase project for push and crash reports, one static dashboard host. Environments are separated by keys, not by code. Database changes are SQL migration files in the repository, applied by the Supabase CLI from CI. Full tool choices are in `tooling.md`.

---

11. Document acceptance criteria

This document passes review when it meets the checklist for `architecture.md` in `loop.md` §3.